PCB4 Anomaly Monitoring Implementation and Evaluation

Group 1

October 23, 2026

Author: Group 1 · Computer Science and Engineering · **Project:** Industrial AI Monitoring Framework

Abstract

This report presents the implementation and evaluation of a local image-based anomaly monitoring prototype for printed circuit boards. The system integrates a Python inference service, a Node-RED workflow, transactional SQLite storage, and the Inspect web interface. A PatchCore-style baseline fitted on the VisA PCB4 category provides real inference outputs for evaluating the complete image-to-alert pipeline.

Model performance and workflow correctness are evaluated separately. The baseline achieves an image-level AUROC of 94.11%; however, at the selected operating threshold, it detects only 31 of 100 anomalous test images, with no false alarms among 101 normal test images. The workflow implements persistent alert storage, duplicate detection, record-conflict handling, and idempotent acknowledgment.

The results demonstrate the feasibility of integrating anomaly inference with a local monitoring workflow. However, the low detection recall and incomplete operational validation limit the prototype's suitability for automated quality inspection.

1. Objectives and Individual Contribution

The objective of this work is to implement and evaluate a reusable model-to-workflow integration for local anomaly monitoring. The intended processing sequence consists of image input, model inference, threshold-based decision making, event creation, and presentation through a local web interface.

The individual contribution focuses on the following components:

- Defining the HTTP input and output contract between the model service and the workflow.
- Implementing the Python inference adapter and Node-RED routing, validation, and threshold logic.
- Preparing and fitting a local VisA PCB4 baseline to replace synthetic scores with real inference outputs.
- Implementing alert creation, duplicate and conflict handling, acknowledgment, and SQLite persistence.
- Developing the Inspect interface, deployment safeguards, and verification scripts.

Development proceeded in two stages. A synthetic-data workflow was first used to test record validation and alert behavior under controlled conditions. The real-image model was then integrated to evaluate the complete processing pipeline.

The checkpoint and model results presented in this report come from the local experiment conducted for this implementation. Related model-code references and their attribution are provided in the References and Attribution section.

2. Dataset and Experimental Protocol

2.1. Dataset Selection

The experiment uses the **PCB4 category of the VisA dataset**, provided by Amazon Science. This category contains normal and anomalous printed circuit board images and supports the evaluation of image-based anomaly detection.

Data preparation follows the official one-class split defined in `split_csv/1cls.csv`. The original partition contains 904 normal training images and 201 test images, comprising 101 normal images and 100 anomalous images.

2.2. Data Preparation and Validation

Only the PCB4 category and the official split definition were extracted for this experiment. Before model fitting, the image and mask files were checked for readability and consistency with the split definition.

The validation procedure examined 1,105 images and 100 anomaly masks. No unreadable images or empty anomaly masks were detected. The original images have a resolution of 1358×1104 pixels.

The original mask values, ranging from 0 to 7, were preserved. This experiment reports image-level detection results only; the masks were not used to calculate pixel-level performance. The displayed heatmaps should therefore be interpreted as visual explanations rather than validated defect segmentations.

2.3. Fit, Calibration, and Test Partitions

The 904 normal training images were divided into two disjoint subsets using a fixed random seed of 42. A total of 804 images were used to construct the feature memory bank, while the remaining 100 images were reserved for threshold calibration. The official test partition was retained without modification.

Partition	Images	Purpose
Fit: normal	804	Memory-bank construction
Calibration: normal	100	Threshold selection
Test: normal	101	False-alarm evaluation
Test: anomaly	100	Anomaly-detection evaluation
Total	1,105	

No image path was shared between the fit, calibration, and test partitions. The split manifest records the image membership of each partition and the SHA256 hash of the official split file to support traceability.

2.4. Evaluation Procedure

Model fitting used only the normal images in the fit partition. The operating threshold was then selected from the scores of the normal calibration images. Neither test labels nor test scores were used to select the initial threshold.

After fixing the model and threshold, all 201 test images were evaluated. Image-level AUROC was used to assess anomaly-score ranking. Precision, recall, F1 score, and confusion counts were calculated at the selected operating threshold.

Per-image predictions were retained to support error analysis, including the inspection of missed anomalies and false alarms. The split and evaluation records are available in the experimental evidence directory.

2.5. Evaluation Limitations

The evaluation is limited to one dataset category and does not establish performance on live camera images, other PCB designs, or different acquisition conditions.

The baseline test results have now been inspected. Consequently, subsequent model or threshold selection should use a separate validation protocol. Improvements selected using the same test results must be identified as exploratory rather than presented as an independent final evaluation.

3. Baseline Method and Environment

3.1. Feature Extraction and Memory Bank

The implementation uses a PatchCore-style anomaly detection baseline with a pretrained ResNet18 backbone. Features are extracted from `layer2` and `layer3`, with the number of neighbors configured as nine.

Input images are resized to 256×256 pixels and normalized once using ImageNet channel statistics. The backbone weights remain fixed: fitting extracts features from normal images and constructs a reference memory bank, rather than updating the backbone through backpropagation.

To limit memory consumption, the implementation uses seeded random-priority sampling to retain at most 2,000 patch-feature vectors. Images are processed in batches of eight during memory-bank construction.

This sampling strategy differs from the greedy coreset selection used in canonical PatchCore. The evaluated method is therefore described as a **PatchCore-style baseline with a bounded random memory bank**.

3.2. Threshold Calibration and Decision Rule

The operating threshold is selected using the anomaly scores of the 100 normal calibration images. Specifically, the implementation computes their 99th percentile using the `higher` quantile method.

For the recorded baseline, the resulting threshold is

$$\tau = 14.64073657989502.$$

Given an image anomaly score $s(x)$, the decision rule is

$$\hat{y}(x) = \begin{cases} 1, & s(x) \geq \tau, \\ 0, & s(x) < \tau, \end{cases}$$

where $\hat{y}(x) = 1$ denotes an anomaly alert and $\hat{y}(x) = 0$ denotes a below-threshold result.

The score represents an anomaly distance rather than a calibrated probability of a defect. Similarly, selecting the 99th percentile of normal calibration scores does not guarantee a 1% false-alarm rate on future observations. The threshold must be evaluated on data representative of the intended operating conditions.

3.3. Experimental Environment

The baseline was executed in the following environment:

Component	Recorded Configuration
Operating system	Windows
Python	3.11.9
PyTorch	`2.6.0+cu124`
torchvision	`0.21.0+cu124`
anomalib	2.2.0
GPU	NVIDIA GeForce RTX 4060
Fit batch size	8 images
Evaluation batch size	1 image
Random seed	42

The model is saved as a custom `model.pt` checkpoint containing its state, including the memory bank, and the configuration required for reconstruction. The inference service loads this checkpoint together with the corresponding `threshold.json` file.

This checkpoint format is specific to the implemented loader and should not be treated as interchangeable with a Lightning `.ckpt` file.

3.4. Methodological Limitations

Resizing the original 1358×1104 images to 256×256 changes their aspect ratio and may reduce the visibility of small defects. A randomly sampled memory bank may also represent normal appearance differently from a greedy coreset.

These are potential factors for further investigation. Their individual effects have not been isolated through controlled experiments, and they should not be presented as established explanations for the observed detection errors.

4. Model Results and Error Analysis

4.1. Evaluation Results

The evaluated run is identified as `visa-pcb4-20261022-213717`. The following results are recorded in the repository's baseline results file.

Metric	Observed Value
Image-level AUROC	94.11%
Precision at the selected threshold	100.00%
Recall at the selected threshold	31.00%
F1 score at the selected threshold	47.33%

The corresponding confusion counts are shown below. An anomalous image is treated as the positive class.

Ground Truth	Below Threshold	Alert	Total
Normal	101 (TN)	0 (FP)	101
Anomaly	69 (FN)	31 (TP)	100
Total	170	31	201

Although the image-level AUROC is 94.11%, the selected operating threshold detects only 31 of the 100 anomalous images. AUROC measures ranking performance across thresholds; it does not represent the proportion of anomalies detected at the deployed threshold.

The absence of false positives in the 101 normal test images produces a precision of 100% for this experiment. This observation does not guarantee equivalent performance on future data. In particular, the 69 missed anomalies show that a below-threshold result cannot be interpreted as evidence that a PCB is defect-free.

4.2. Representative Predictions

Checkpoint reload verification reproduced the baseline scores for the following selected images. Paths are relative to `pcb4/Data/Images`.

Image	Raw Score	Result
`Normal/0011.JPG`	12.4063	Correctly below threshold
`Anomaly/009.JPG`	14.8036	Correct anomaly alert
`Anomaly/000.JPG`	12.5620	Missed anomaly

These examples illustrate a correct normal decision, a detected anomaly, and a false negative at the operating threshold of approximately 14.6407. They are illustrative cases rather than a separate evaluation dataset.

4.3. Processing Time

Measurement	Observed Value
Memory-bank fitting time	17.17 s
Median inference time	25.03 ms/image
95th-percentile inference time	27.39 ms/image
Median image-to-score time	42.39 ms/image

Inference timing was measured with batch size one after warm-up, with GPU completion accounted for. Image-to-score timing additionally includes image loading and preprocessing.

These measurements do not represent complete web-request latency. The interactive application also performs heatmap and overlay generation, HTTP communication, and database operations. Its end-to-end latency must therefore be measured and reported separately.

4.4. Error Analysis and Interpretation

The main observed limitation is the high number of missed anomalies at the selected threshold. The calibration policy, loss of image detail during resizing, and the representativeness of the bounded memory bank are potential factors for investigation.

Their effects have not been separated through controlled experiments. Consequently, they are hypotheses rather than established explanations for the detection errors. No model-improvement run has been completed after this baseline.

Future experiments should use a defined validation protocol to evaluate changes in threshold selection, image resolution, or memory-bank construction. Improvements should not be selected using the inspected test set and then reported as an independent final evaluation.

5. Integrated Architecture and Data Contract

5.1. Processing Sequence

The application separates image inference from workflow coordination and persistent state management.

1. The browser submits a JPEG or PNG image encoded as base64, together with a record identifier and a UTC timestamp.
2. Node-RED validates the request and calls the local Python inference service.
3. The model service returns the anomaly score, threshold, model identifier, image fingerprint, processing time, and visualization outputs.
4. Node-RED validates the model response and applies the threshold-based decision rule.
5. The decision is submitted to the SQLite-backed storage component for transactional processing.
6. After successful persistence, the result is returned to the browser for display.

Inference, validation, communication, and storage failures are returned as errors. A failed processing step does not produce a normal classification. A successful result is returned only after the storage operation has committed.

5.2. Interface Contract

Boundary	Contract or Responsibility
Browser input	`image_base64`, `record_id`, and `timestamp`; one JPEG/PNG image up to 5 MB.
Model response	Raw anomaly score and threshold, model version, image SHA256, processing time, heatmap, and overlay.
Workflow decision	`alert` when the raw score meets or exceeds the threshold; otherwise `normal`.
Persistent storage	Canonical payload fingerprint, record result, alert state, and source timestamp watermark.
Browser output	Decision, raw score, threshold, timing, image comparison, and alert status.

The timestamp records submission time. It is not a verified camera acquisition timestamp. This distinction must be addressed before extending the application to live-camera or sensor synchronization.

5.3. Local API and Execution Scope

The local demo API uses the prefix `/giang-pcb4/v1`.

Method	Route	Purpose
GET	`/demo`	Serve the Inspect interface
POST	`/infer`	Submit an image for inspection
GET	`/health`	Read model readiness information
GET	`/alerts`	Retrieve alerts and counts
POST	`/ack`	Acknowledge an alert

The Python service binds to `127.0.0.1:1891`, and the Node-RED workflow checks that requests originate from loopback clients.

Inference requests are processed serially. The current prototype targets local, single-user inspection with small requests; concurrent multi-camera processing has not been evaluated.

5.4. Score Adaptation and Visualization

The raw model score is an anomaly distance and is not restricted to the interval $[0, 1]$. For compatibility with the workflow representation, the adapter computes

$$q = \frac{s}{s + \tau},$$

where $s \geq 0$ is the raw score and $\tau > 0$ is the raw threshold.

The transformed decision boundary is 0.5:

$$q \geq 0.5 \quad \Longleftrightarrow \quad s \geq \tau.$$

This transformation preserves the threshold decision but does not calibrate the score into a defect probability. The interface therefore prioritizes the raw score and threshold when presenting an inspection result.

Heatmaps are normalized separately for each image to visualize relative anomaly intensity. Their colors should not be interpreted as calibrated probabilities or compared directly across images. Pixel-level localization accuracy has not been evaluated.

6. Alert Lifecycle and Persistence

6.1. Alert Creation and Acknowledgment

A valid inspection result is stored regardless of whether it exceeds the anomaly threshold. An alert is created only when the score meets or exceeds that threshold.

Each alert has one of two states:

- `pending`: the alert is awaiting review.
- `acknowledged`: a user has acknowledged the alert.

Acknowledgment records that an alert has been reviewed. It does not establish that the detected defect has been repaired or that the inspected PCB is acceptable.

The implemented behavior is summarized below.

Condition	**System Behavior**
Valid below-threshold result	Store the result without creating an alert.
Score meets or exceeds threshold	Create one `pending` alert.
Identical record retry	Return a duplicate result without creating an additional alert.
Existing ID with changed content	Return a conflict error.
First acknowledgment	Change the alert to `acknowledged` and record the acknowledgment time.
Repeated or concurrent acknowledgment	Apply the state change only once and preserve the first acknowledgment time.
Unknown alert identifier	Return an explicit not-found error.
Invalid input or processing failure	Return an error rather than a normal decision.
Storage capacity reached	Reject operations that would exceed the limit without silently deleting pending alerts.

6.2. Duplicate Detection and Temporal Validation

The storage component retains a canonical payload fingerprint alongside each record. A retry with an existing identifier and identical content is recognized as a duplicate. Reusing that identifier with different content produces a conflict.

The workflow also checks record freshness and ordering. A source timestamp watermark tracks the accepted temporal progression of records. Records outside the freshness allowance or arriving out of order are rejected.

Duplicate detection is performed before the staleness check. This allows an identical retry of a retained record to return its previous result without being treated as a new observation.

6.3. Persistent Storage and Capacity

SQLite stores inspection records, payload fingerprints, alerts, acknowledgment timestamps, and the source watermark. Related state changes are processed transactionally to avoid partially applied operations.

The configured limits are:

- 20,000 stored alerts.
- 100,000 stored inspection records.

No automatic time-to-live expiration or archival process is implemented. Acknowledging an alert changes its state but does not remove it or release storage capacity.

When a capacity limit would be exceeded, the operation returns an error. Pending alerts are not silently evicted to accommodate new results. Long-term operation therefore requires an explicit retention and archival policy.

6.4. Migration from the Earlier RAM-Based Workflow

The earlier workflow stored alert state in memory. Migration to the SQLite version preserves exported alerts and their acknowledgment state.

Some legacy records do not contain the complete image fingerprint required by the newer duplicate-detection contract. Their identifiers remain reserved, and a retry returns a dedicated error rather than assuming that the submitted content matches the original record.

The previous normal-record deduplication cache is not migrated. In addition, startup refuses to create an empty replacement database when an existing persistent flow indicates that its database is missing.

These behaviors make incomplete migration or missing storage visible rather than silently presenting an empty alert history.

6.5. Persistence Verification and Remaining Limits

Tests have covered database reopening, backup creation, transaction rollback, and concurrent retry and acknowledgment behavior.

However, a complete stop-and-restart recovery exercise for the running model service and Node-RED workflow remains outstanding. Passing database tests does not establish recovery of the entire application after a process failure or system crash.

7. Interface and Deployment

7.1. Interface Organization

The Inspect interface separates three tasks: image inspection, alert management, and review of model information.

View	Main Functions
Image inspection	Image selection or drag-and-drop upload, original-image and heatmap comparison, overlay selection, zoom, score display, and inspection timing.
Alert management	Total and per-state counts, status filtering, search, pagination, and acknowledgment.
Model information	Current configuration, recorded evaluation results, score interpretation, and known limitations.

The interface also supports retrying the same inspection record and exporting results as JSON. Text labels and icons accompany color-coded states so that color is not the only indication of a result.

Result guidance distinguishes a below-threshold decision from a confirmed defect-free product. Model limitations are available in the information view.

7.2. Source and Workflow Consistency

The Node-RED import file is generated from source, including the HTML served by the workflow. Changes to the interface must therefore be followed by flow regeneration or use of the UI update script.

Maintaining the generator as the source of truth prevents a later build from discarding changes made only to an exported flow.

The English documentation is maintained separately from interface deployment. English interface text has been prepared, but deployment of the translated interface has not been verified. Documentation translation should therefore not be interpreted as evidence that the running application has been updated.

Presentation changes retain the existing API identifiers, record fields, and decision semantics.

7.3. Deployment Safeguards

The deployment scripts include the following safeguards:

- Backing up existing flows and, where applicable, workflow storage before deployment.

- Checking node identifiers and HTTP routes for collisions.
- Using the current Node-RED revision to detect conflicting updates.
- Restricting changes to the intended demo components and checking that unrelated flows remain unchanged.
- Comparing generated source with the deployed definitions.

For interface-only changes, the UI updater replaces the HTML-serving function without restarting the Python inference service. It checks the expected flow structure before updating and verifies source matching, model readiness, and preservation of existing alerts afterward.

7.4. Verification Scope

Recorded deployment checks confirm selected properties such as source consistency, model readiness, and preservation of alert state. They do not constitute complete browser interaction coverage.

The redesigned interface includes upload, zoom, filtering, search, pagination, and export controls, but comprehensive interaction testing remains outstanding.

Published verification files describe their original runs. Translating interface text or test descriptions does not constitute a new HTTP test run, and historical results should not be relabeled as verification of a later deployment.

8. Verification and Evidence

8.1. Verification Strategy

Verification was performed at several levels to distinguish model behavior, storage correctness, HTTP integration, and interface deployment.

A passing result at one level does not establish correctness at every other level. For example, reproducing a model score does not verify HTTP communication, and reopening a database does not demonstrate recovery of the complete application after a process failure.

8.2. Recorded Results

Verification Layer	Recorded Result	Evidence and Scope
Local checkpoint	Three selected scores reproduced; invalid image rejected.	Local verification. Checks selected predictions after loading the checkpoint; does not exercise HTTP.
SQLite storage	Seven test groups passed.	Storage test suite. Covers concurrent retry and acknowledgment, capacity, migration, backup, database reopening, and rollback.
Real-model Python HTTP	Recorded integration checks passed.	HTTP verification. Includes readiness, decisions, duplicate handling, acknowledgment, and intentional invalid inputs.
Interface deployment	Source matched; alerts preserved; model ready.	Deployment verification. Does not provide complete browser interaction coverage.
Synthetic engine	15 checks passed.	Offline validation of record-processing rules.
Synthetic workflow	98 checks passed.	Offline workflow checks, including simulated failure and retry scenarios.
Synthetic Python unit tests	Four tests passed.	Client-side unit checks, separate from HTTP and image inference.
Synthetic Python HTTP	93 checks passed.	Recorded requests against the synthetic HTTP workflow.

Verification Layer	Recorded Result	Evidence and Scope
Synthetic browser HTTP	94 checks reported in a captured screenshot.	No exported browser-v2 JSON is included in the shared evidence package.
Full service restart	Not completed.	Requires an explicit stop/start exercise and comparison of retained application state.

The synthetic-workflow result files are available in the legacy evidence directory.

The difference between the Python and browser HTTP check counts reflects assertion grouping: the Python suite combines acknowledgment checks that the browser suite records separately.

8.3. Expected Errors and Test Provenance

Tests intentionally submit malformed data, unknown identifiers, and other invalid inputs. Such a case passes only when the system returns the expected error and preserves the required state. An error response alone is not sufficient evidence of correct behavior.

Successful recorded HTTP checks were executed from the local Windows terminal and browser. Other execution attempts encountered socket restrictions, including `WinError 10013`. Those attempts are classified as blocked tests rather than successful end-to-end verification.

Verification files retain the scope of their recorded runs. Subsequent documentation or interface changes do not automatically inherit those passing results.

8.4. Synthetic HTTP Load Test

The earlier workflow was tested with 10,000 valid synthetic records from ten sources. Requests contained batches of 100 records and were sent with concurrency one across three bounded load stages.

Target Rate (records/s)	Records	Duration (s)	Observed Rate (records/s)
100	1,000	9.01	111.03
500	3,000	5.80	516.88
1,000	6,000	5.91	1,015.88

All records were accepted in the recorded run without unexpected errors. The immediate first batch and the measurement interval explain why observed rates slightly exceeded the configured targets.

These measurements describe HTTP workflow processing of synthetic scores. They do not measure AI image inference throughput, maximum system capacity, or long-duration stability.

9. Remaining Work and Project Milestones

9.1. Progress Against Project Requirements

The current implementation addresses the initial model-to-workflow integration milestone. Further evidence is required before claiming completion of the broader project requirements.

Milestone	Current Status	Required Next Step
Dataset and baseline	Completed for VisA PCB4.	Improve detection recall using a defined validation protocol and documented error analysis.
Image-to-event-to-web integration	Demonstrated locally.	Complete redesigned-interface interaction tests and full service recovery checks.

Milestone	Current Status	Required Next Step
Reusable model block	HTTP adapter implemented.	Externalize model and dataset configuration; extend model-timeout and service-failure tests.
Threshold configuration and workflow reload	File-based threshold and generated flow available.	Verify a controlled threshold change and import the workflow into a clean local setup.
Comparison of two workflow tools	Not completed.	Process identical images through the same model using a second feasible tool and compare results under equivalent conditions.
Offline operation	Local checkpoint loading implemented.	Package dependencies and required model assets, then test with Internet access unavailable.
Sensor integration	Synthetic HTTP groundwork completed.	Define the sensor contract, test MQTT locally, and integrate a real device after validating the model block.

9.2. Recommended Implementation Order

The next iteration should prioritize the following work:

1. **Complete operational verification.** Test the redesigned interface, service stop/start, retained alert state, and model-unavailable behavior.
2. **Improve the model through controlled experiments.** Define validation data and a detection objective before changing threshold selection, image resolution, or memory-bank construction. Record both missed anomalies and false alarms.
3. **Compare workflow alternatives.** Use the same model, images, and decision rules when evaluating a second orchestration tool. Measure latency, error visibility, integration effort, and reproducibility.
4. **Extend the validated pipeline to sensors.** Introduce a common sensor record format and a simulated local source before connecting real hardware.

The inspected 201-image test set should not be reused for model selection and then presented as an independent final evaluation. Results obtained through such tuning must be labeled as exploratory.

Similarly, a working image-upload application does not establish performance with live cameras or sensors. Those extensions require their own data-quality, timing, connectivity, and recovery tests.

10. Conclusion

This work implemented a local anomaly-monitoring prototype that connects real image inference, Node-RED decision logic, SQLite-backed alert state, and the Inspect web interface. The system supports duplicate detection, record-conflict handling, and idempotent acknowledgment, with verification performed at model, storage, and HTTP levels.

The strongest current evidence concerns integration and alert-processing behavior. Although the PCB4 baseline achieved an image-level AUROC of 94.11%, its recall at the selected threshold was only 31%. The model is therefore not sufficiently reliable for automatic product acceptance.

The implementation provides a practical foundation for further experimentation. The next stage is to improve detection performance through controlled validation, complete operational recovery and interface tests, compare workflow alternatives, and then extend the validated pipeline to physical devices.

Source code, configuration, workflow definitions, and recorded evidence are provided in the repository to support review and reproduction of the reported work.

References and Attribution

1. **Project implementation.** Tran Long Giang. *Industrial AI Monitoring Framework*. Source code, setup instructions, workflow definitions, and experimental evidence for the implementation described in this report.

2. **Dataset.** Amazon Science. *VisA: Visual Anomaly Dataset*. Source of the PCB4 images and official data split used in this experiment. Dataset images are not redistributed with this project; users should consult the accompanying license and attribution terms.
3. **Related model implementation.** Bao. *AIoT_H261: Group Model Baseline*. Reference implementation consulted during model integration. Its published benchmark results are not presented as reproduced results in this report.
4. **Project requirements.** Supervisor's project brief supplied to Group 1. Requirements for a reusable model block, workflow comparison, local monitoring, and subsequent sensor integration. Internal course material.

Contribution Statement

The work reported here covers the locally fitted PCB4 baseline, the inference-service adapter, Node-RED integration, alert lifecycle, SQLite persistence, the Inspect interface, and verification procedures. The evaluated checkpoint was fitted locally by Tran Long Giang; it was not a pretrained checkpoint supplied by the related group repository.

Source Code and Supporting Evidence

Reproduction guidance and recorded results are available in the project repository:

- Demo setup and operation
- Dataset preparation and model fitting
- Experimental evidence and interpretation